

CS4023D Artificial Intelligence



Module-I

Structures and Strategies for State Space Search

Defining AI Problem

- To build a system to solve a particular problem, we need to do four things.

1. Define the Problem Precisely:

Should include precise specifications, initial conditions and acceptable solutions.

2. Analyze the Problem:

- a) Isolate and represent the task knowledge that is necessary to solve the problem
- b) Choose the best problem solving techniques and apply to solve the problem.

Defining The Problem as a State Space Search

- A State Space is represented by **four-tuple** $[N, A, S, GD]$ where:
 - N** is the set of states or nodes of the graph.
 - A** is the set of arcs or links between nodes.
 - S** , a nonempty subset of N, contains the start state(s) of the problem.
 - GD**, a nonempty subset of N, contains the goal state(s) of the problem. The states in GD are described using either:
 1. A measurable property of the states encountered in the search.
 2. A property of the path developed in the search, for example, the transition costs for the arcs of the path
- A **Solution path** is a path through this graph from a node in S to a node in GD.
 - Arcs** of the state space correspond to steps in a solution process.

Example

8-Puzzle(or 15-Puzzle)

- In the 8-Puzzle , state is of size 3×3 , with 8 differently numbered tiles fitted into 9 spaces on a grid .

One space is left blank so that the tiles can be moved around to form different patterns .

- ***Possible Moves :***
 - Move the blank UP
 - Move the blank DOWN
 - Move the blank LEFT
 - Move the blank RIGHT

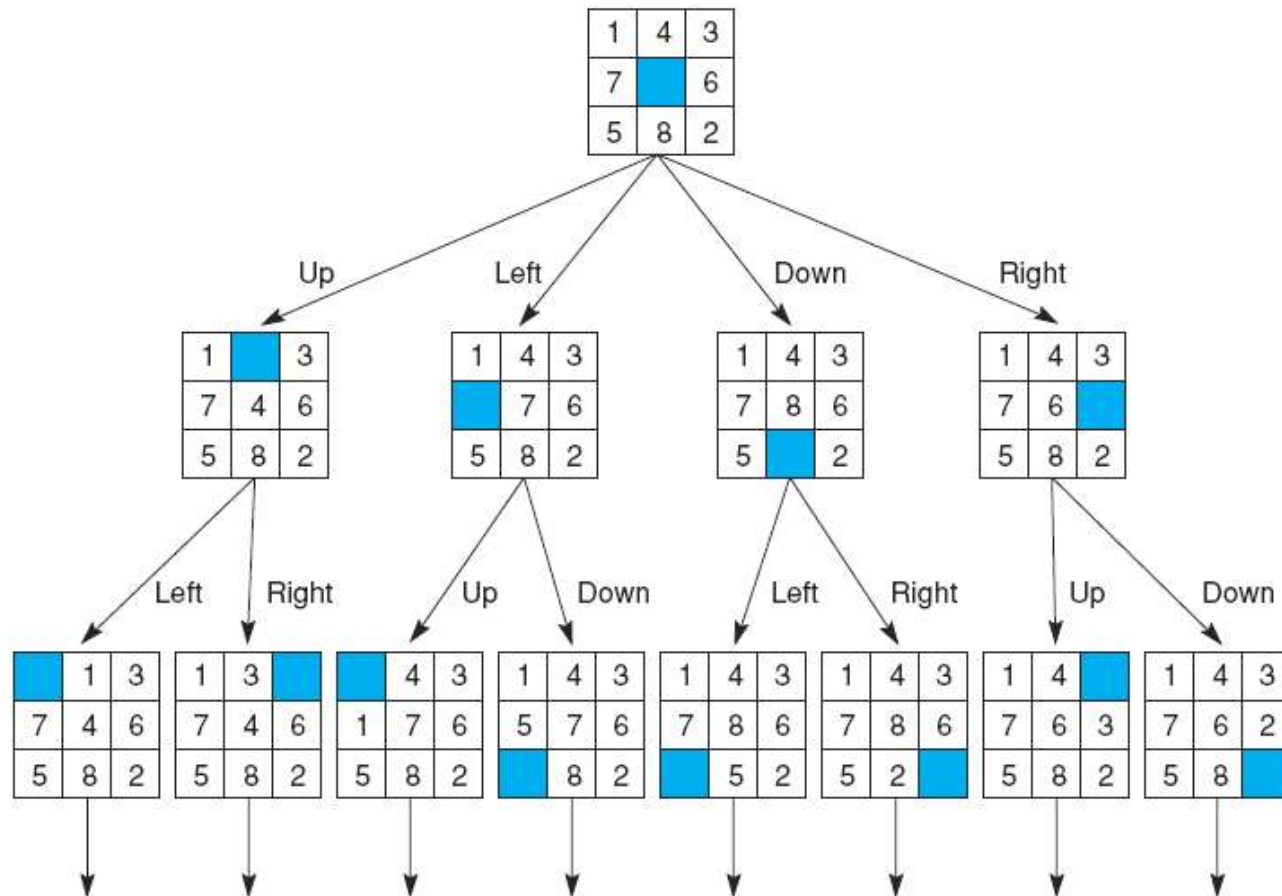
$(n^2 - 1)$ -puzzle

8	2	
3	4	7
5	1	6

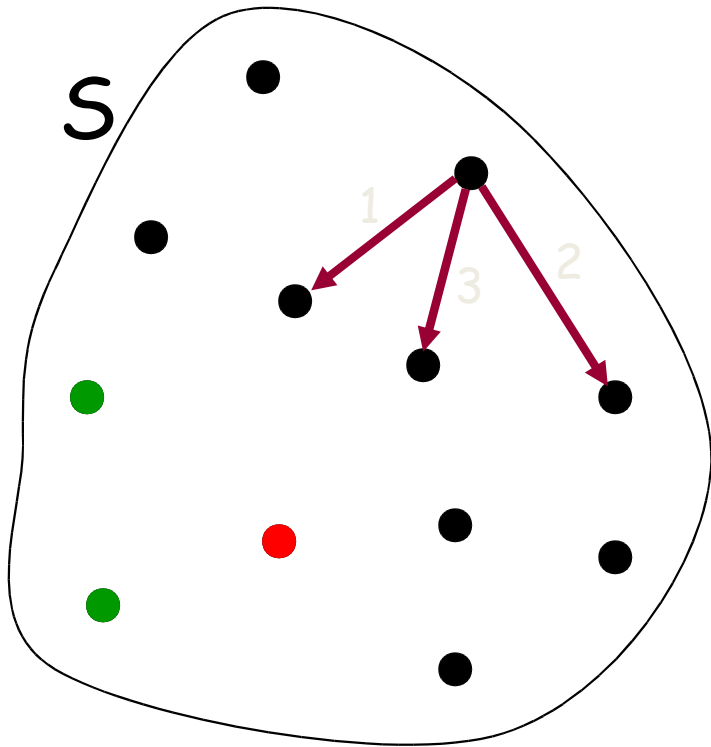
1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	

...

State space of the 8-puzzle generated by “move blank” operations



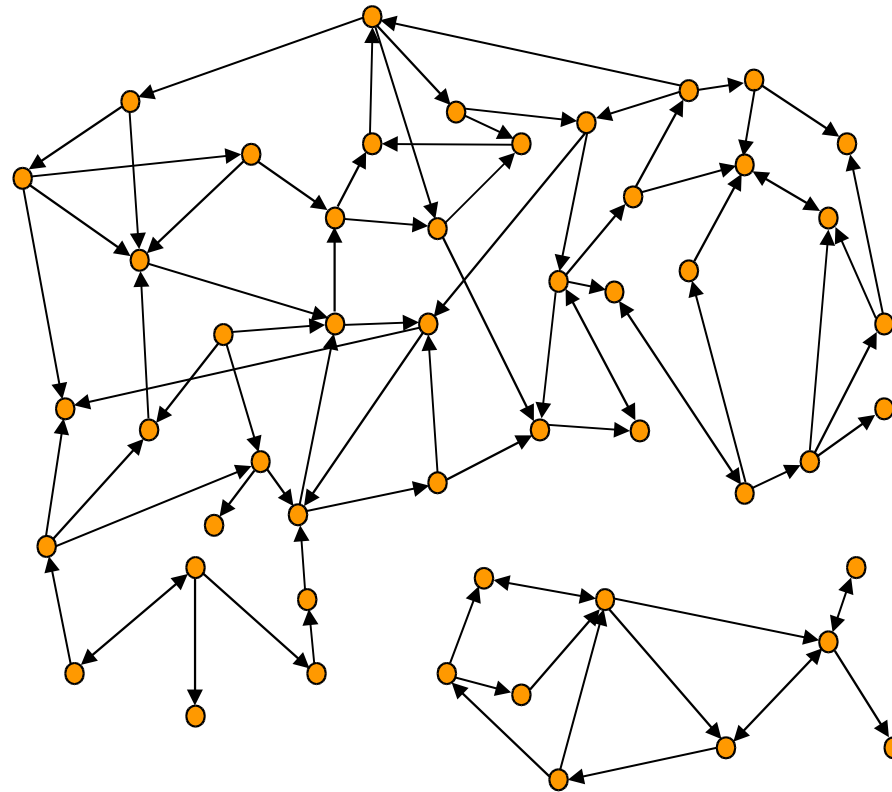
Stating a Problem as a Search Problem



- State space S
- Successor function:
 $x \in S \rightarrow \text{SUCCESSORS}(x) \in 2S$
- Initial state s_0
- Goal test:
 $x \in S \rightarrow \text{GOAL?}(x) = \text{T or F}$
- Arc cost

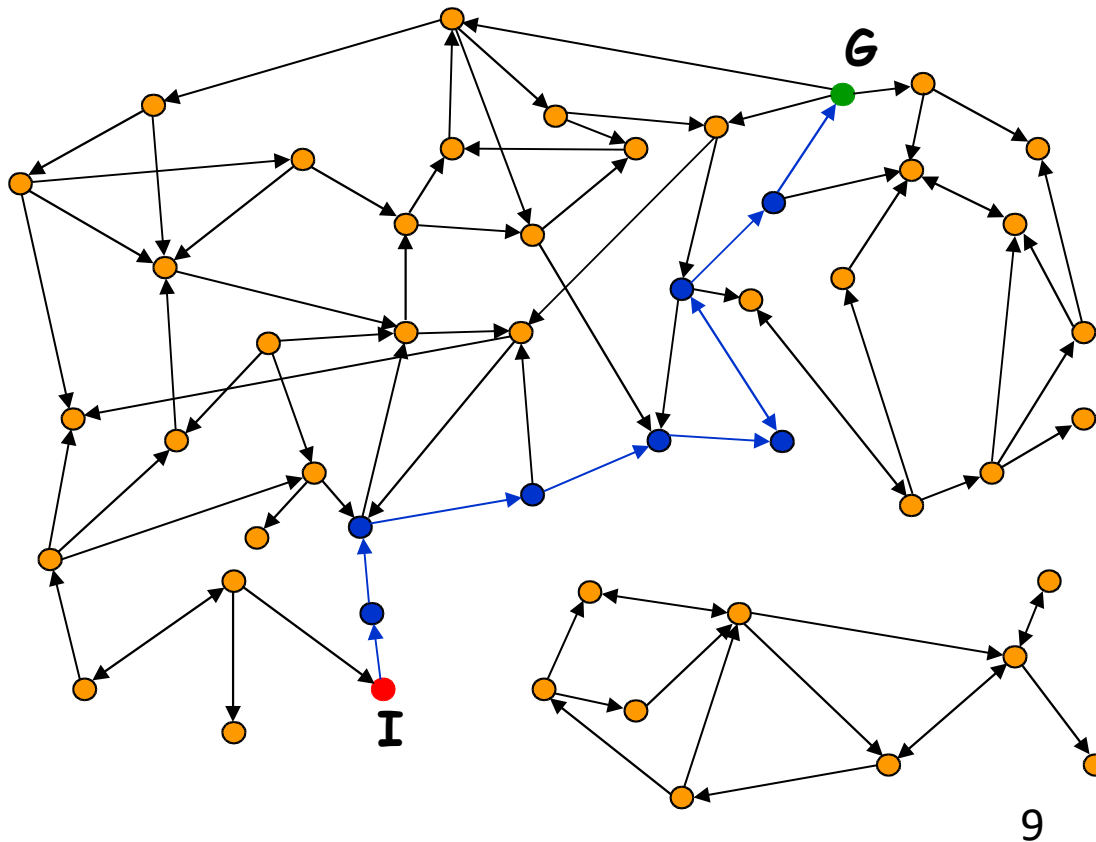
State Graph

- Each state is represented by a distinct node
- An arc (or edge) connects a node s to a node s' if $s' \in \text{SUCCESSIONS}(s)$
- The state graph may contain more than one connected component



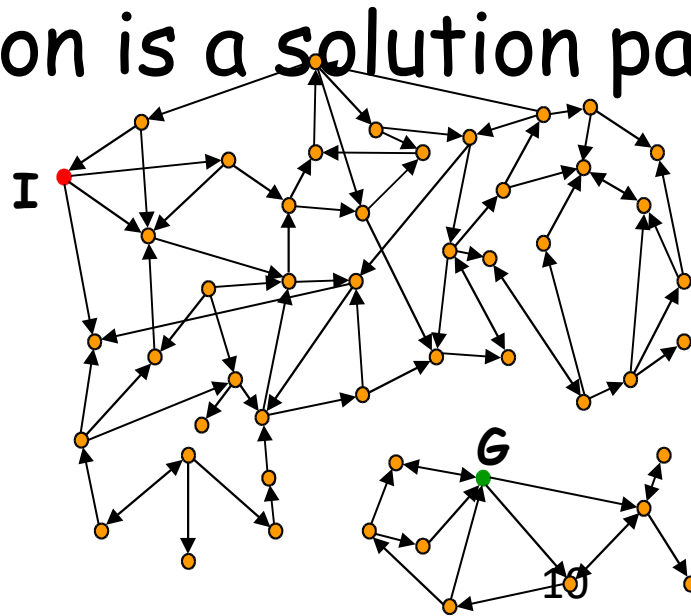
Solution to the Search Problem

- A solution is a path connecting the initial node to a goal node (any one)



Solution to the Search Problem

- A solution is a path connecting the initial node to a goal node (any one)
- The cost of a path is the sum of the arc costs along this path
- An optimal solution is a solution path of minimum cost
- There might be no solution !



How big is the state space of the (n^2-1) -puzzle?

- 8-puzzle $\rightarrow 9! = 362,880$ states
- 15-puzzle $\rightarrow 16! \sim 2.09 \times 10^{13}$ states
- 24-puzzle $\rightarrow 25!$

But only half of these states are reachable from any given state
(but you may not know that in advance)

What is the Actual State Space?

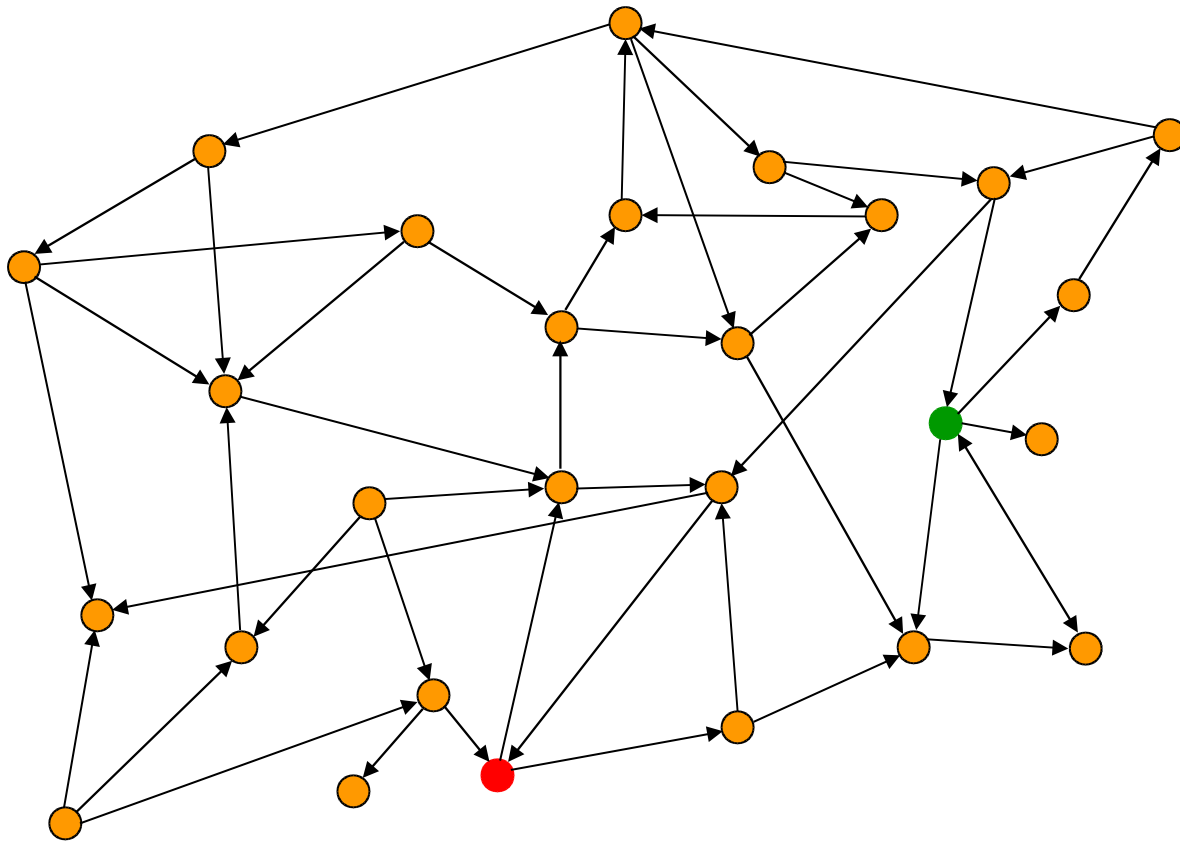
- a) The set of all states?
[e.g., a set of $16!$ states for the 15-puzzle]
- b) The set of all states reachable from a given initial state?
[e.g., a set of $16!/2$ states for the 15-puzzle]

In general, the answer is a)

[because one does not know in advance which states are reachable]

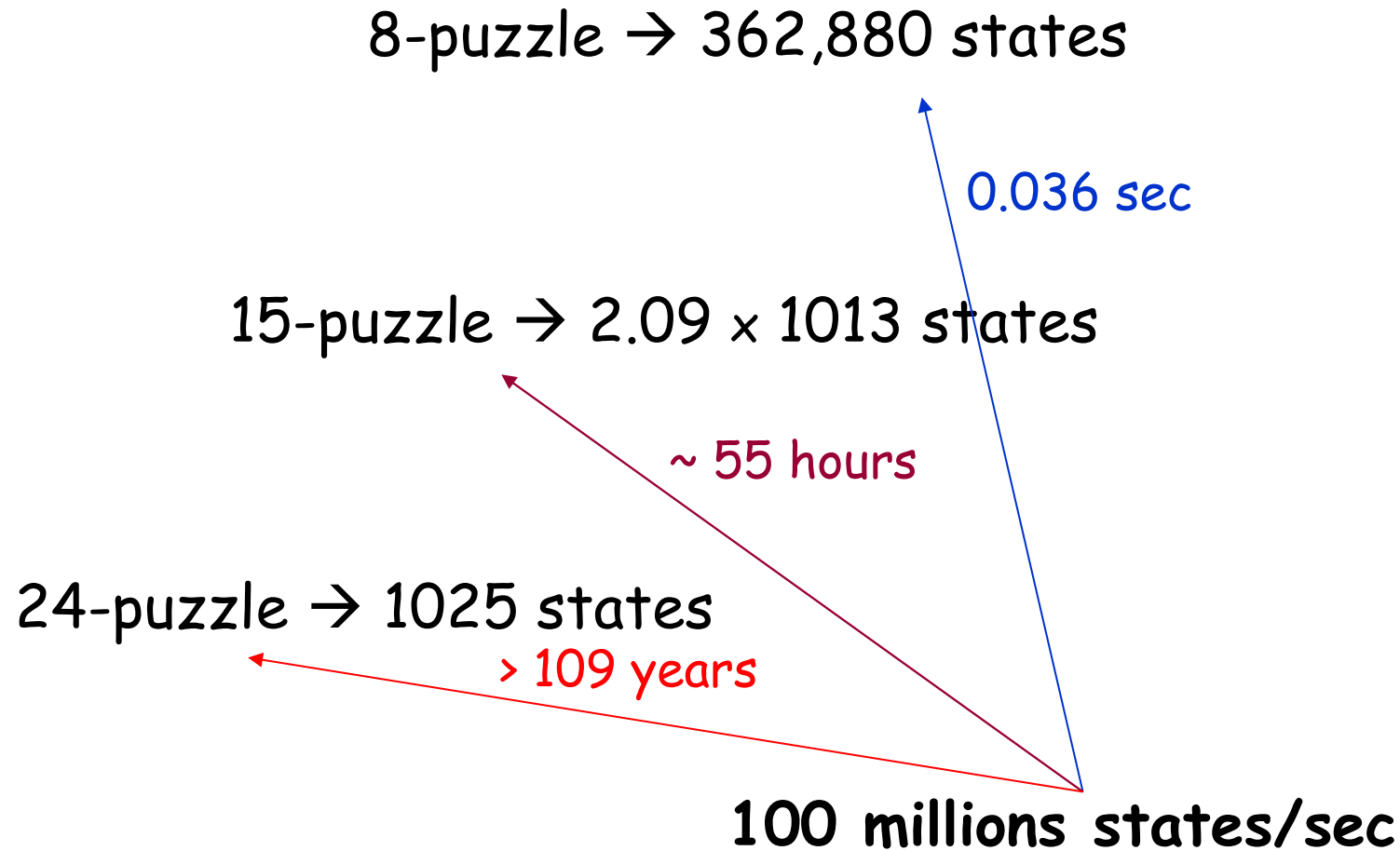
But a fast test determining whether a state is reachable from another is very useful, as search techniques are often **inefficient** when a problem has no solution

Searching the State Space

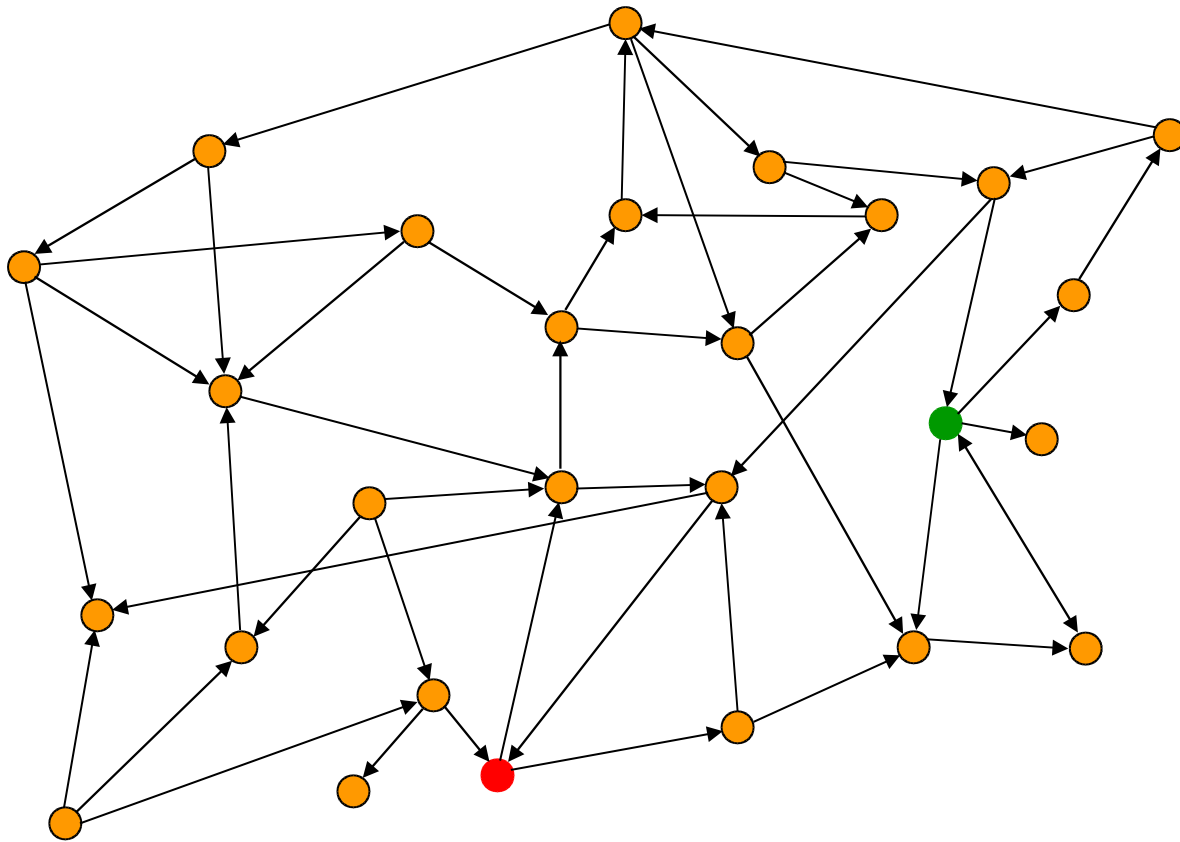


- It is often not feasible (or too expensive) to build a complete representation of the state graph

8-, 15-, 24-Puzzles

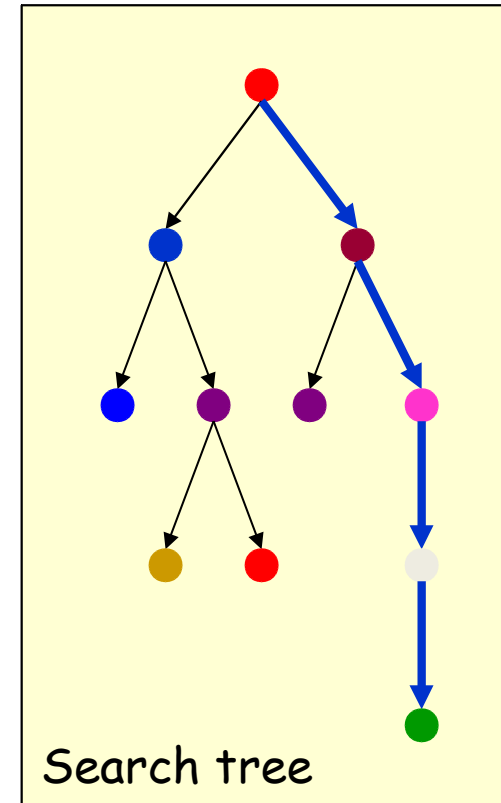
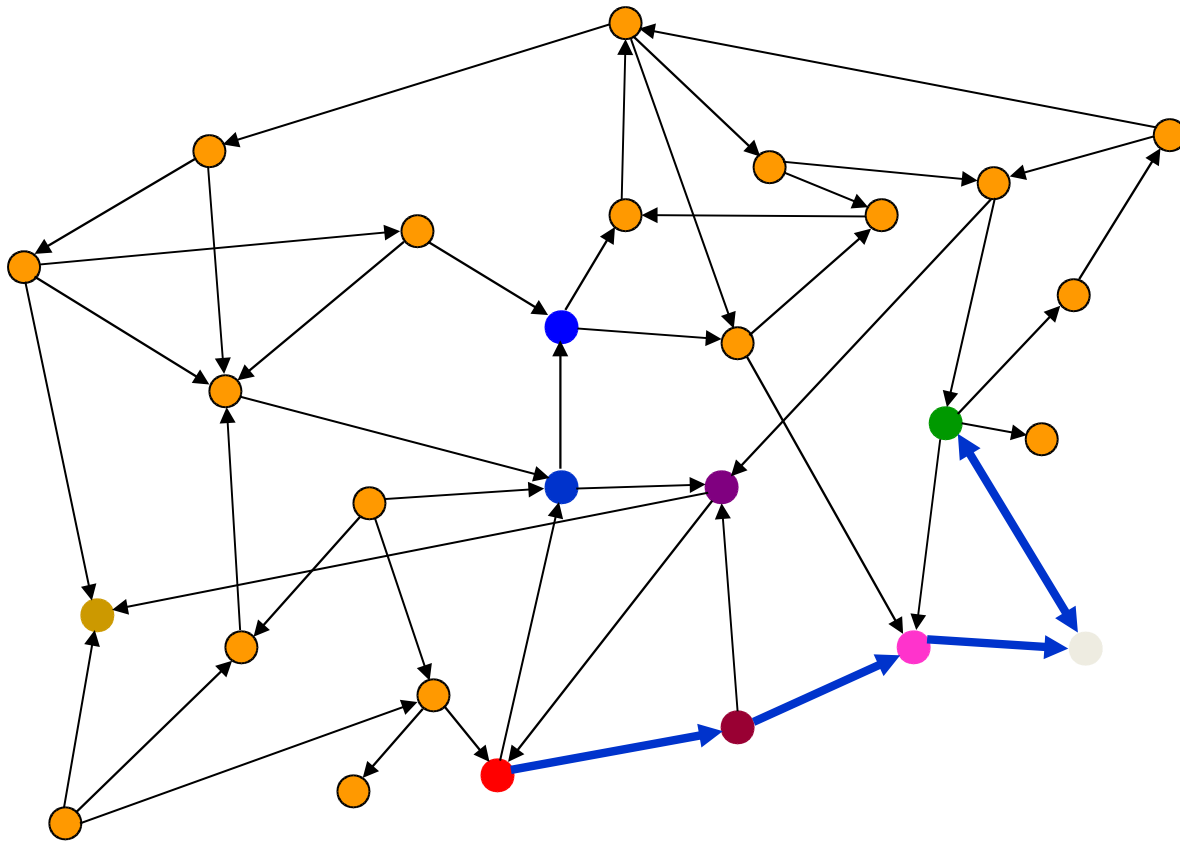


Searching the State Space



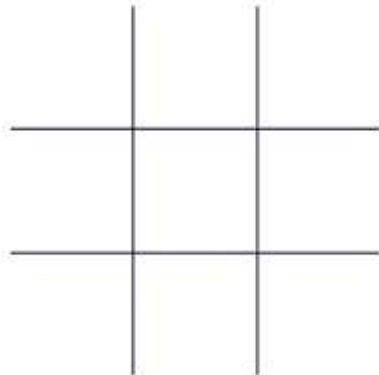
- Often it is not feasible (or too expensive) to build a complete representation of the state graph
- A problem solver must construct a solution by exploring a small portion of the graph

Searching the State Space

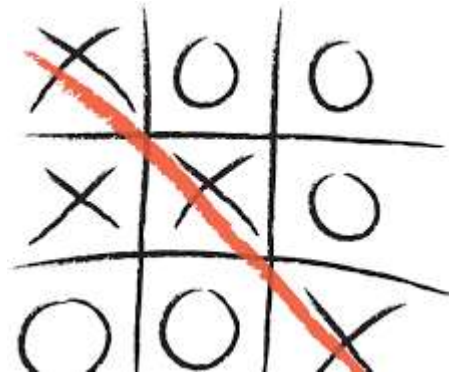


Tic-Tac-Toe

1. The start state is an empty board
2. The termination or goal description is a board state having three X's in a row, column or diagonal.



Start state



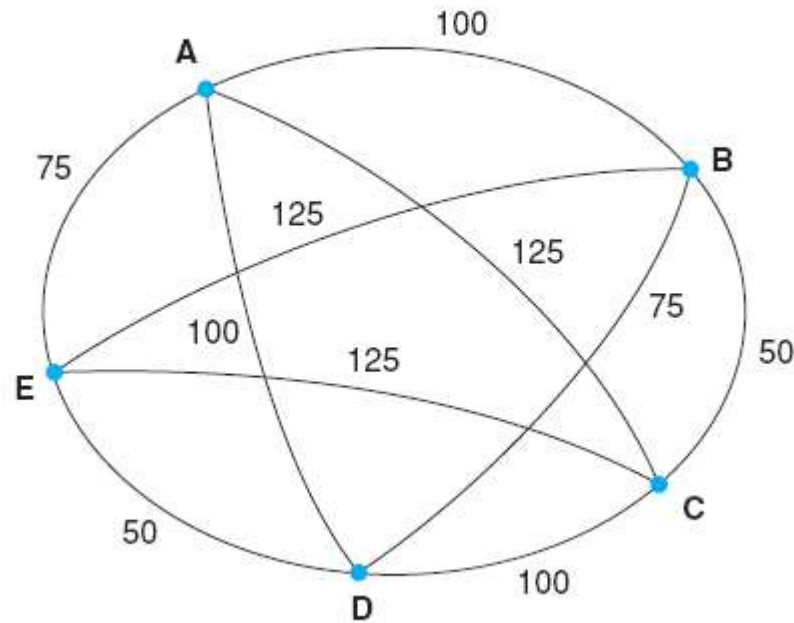
Goal state

3. The path from the start state to a goal state gives the series of moves in a winning game.

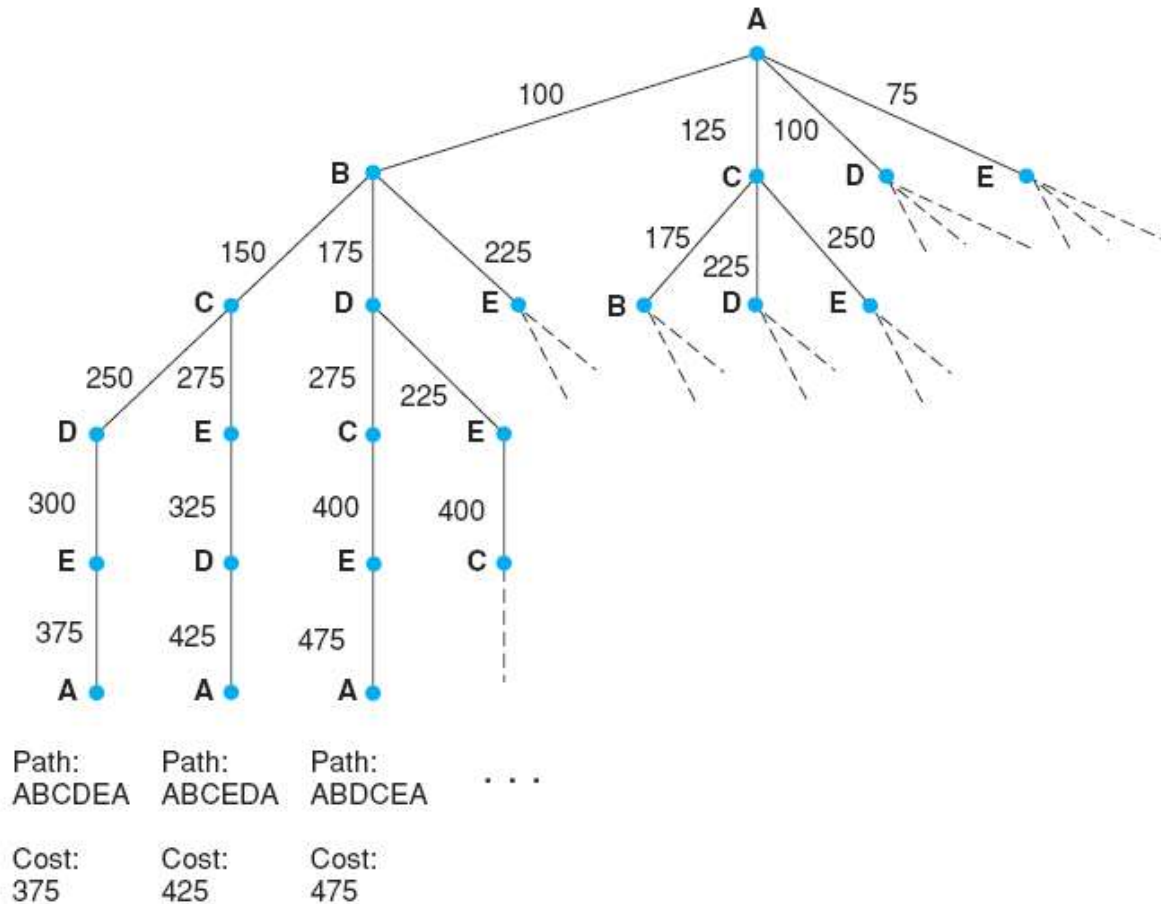
Tic-Tac-Toe

4. There are 3^9 ways to arrange {blank,X,O} in nine spaces, but most of them never occur in an actual game
5. The state space is a tree rather than a graph.
6. There are no cycles in the state space
7. There are nine first moves with eight possible responses to each of them, followed by seven possible responses to each of these and so on. So total $9!$ paths can be generated.

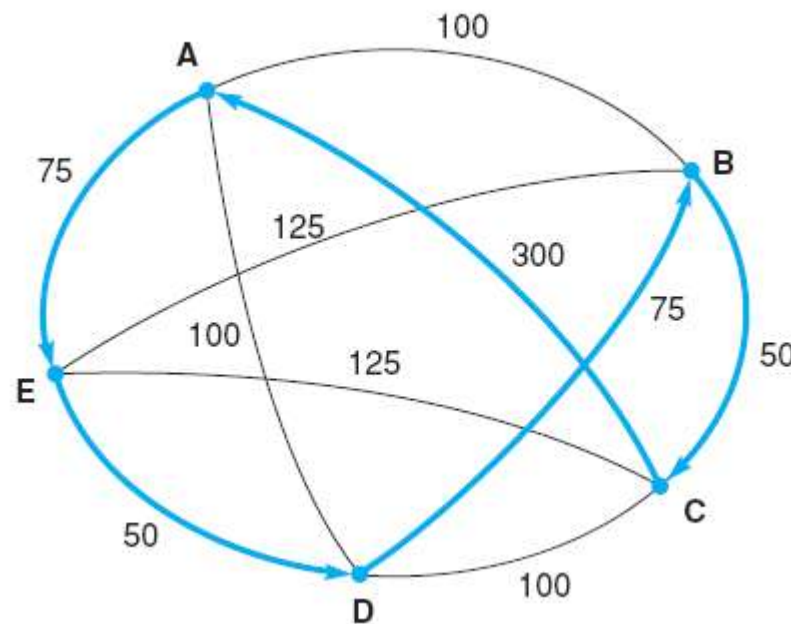
An instance of the travelling salesperson problem



Search for the travelling salesperson problem. Each arc is marked with the total weight of all paths from the start node (A) to its endpoint.

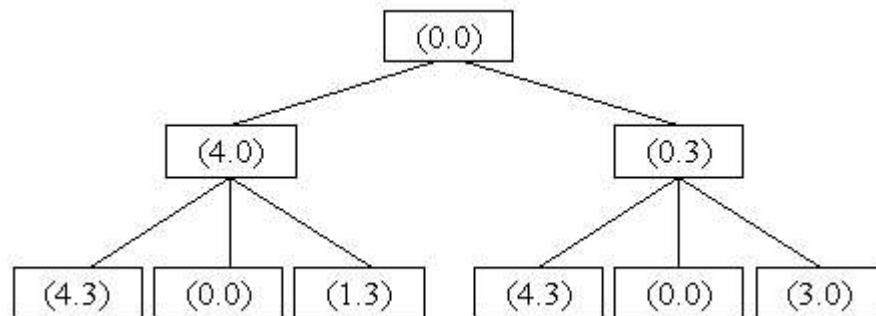


An instance of the travelling salesperson problem with the nearest neighbour path in bold. Note this path (A, E, D, B, C, A), at a cost of 550, is not the shortest path. The comparatively high cost of arc (C, A) defeated the heuristic.



Water Jug-Problem

- You are given with two jugs, a 4-ltr one and a 3-ltr one. Neither has any measuring markers on it. There is pump that can be used to fill the jugs with water. How can you get exactly 2 ltrs water into the 4-ltr jug?



STRATEGIES FOR SPACE SEARCH

- Generally, there are 2 strategies for space search.

1. Data driven search(Forward Chaining) :

The search is driven from the initial state to the final state.

The problem solver begins with the given facts of the problem and a set of legal moves or rules for changing state. Search proceeds by applying rules to facts to produce new facts, which are in turn used by the rules to generate more new facts. This continues till we reach a state that satisfies the goal condition.

2. Goal Driven Search (Backward Chaining):

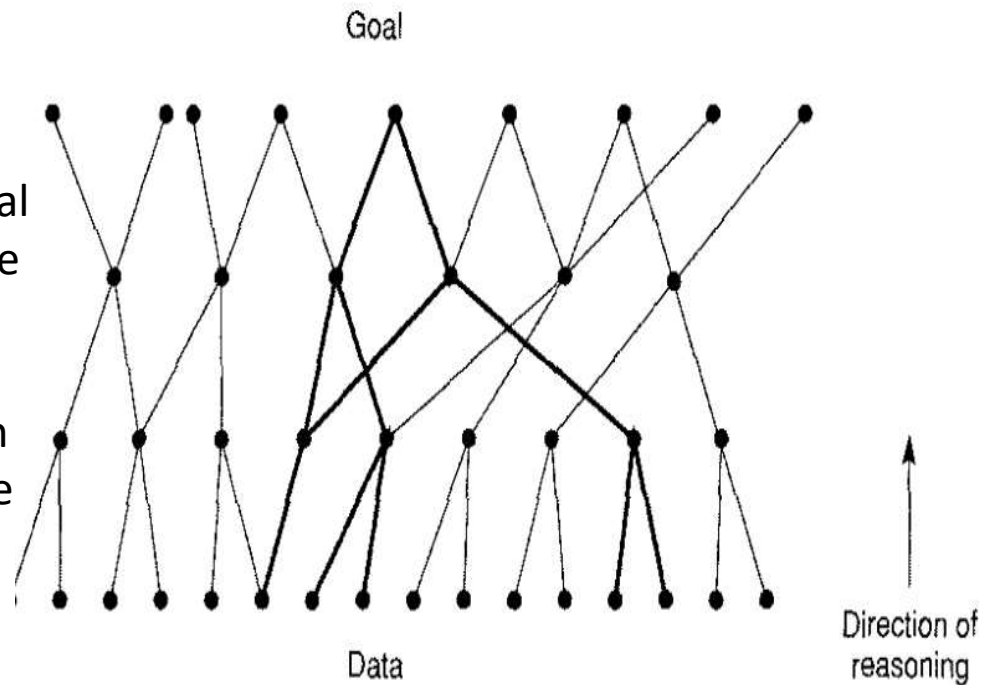
Drives the search from goal state to the starting state.

Here, we try to understand the rules that could be used to generate this goal and determine what conditions must be true to use them. These conditions, become the new goals or subgoals for the search. Search is continued till the initial state is obtained.

Data Driven Search

WHEN TO USE IT??

- (i) All or most of the data are given in the problem statement.
- (ii) There are a large number of potential goals, but there are only few ways to use the facts of the problem instance.
- (iii) When it is difficult to form a goal.
 - eg: If the current configuration in a chess game can lead to a checkmate

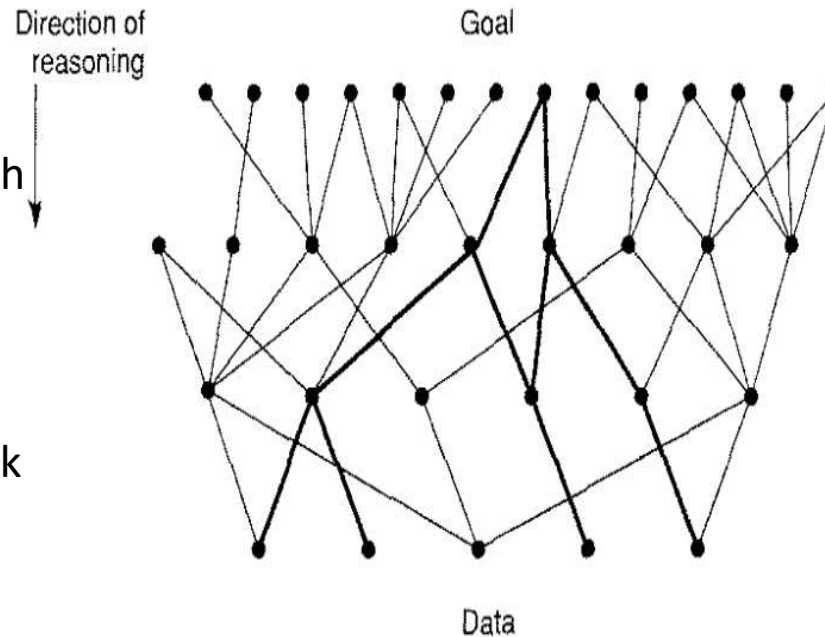


Goal Driven Search

WHEN TO USE IT?

- (i) When the goal is clearly defined.
- (ii) There are large number of rules that match the facts of the problem and thus produce an increasing number of goals.
- (iii). problem data are not given but must be acquired by the solver.

eg: In a family tree problem, to check if person X is a descendant of person Y, its better to use a goal driven search as each person has only 1 parent as opposed to each person having multiple children.



Kapoor Family Tree

Dewan Keshavmal

Bhesharnath Kapoor

b. 1906

Prithviraj Kapoor m Ramsarni (Rama) Mehra

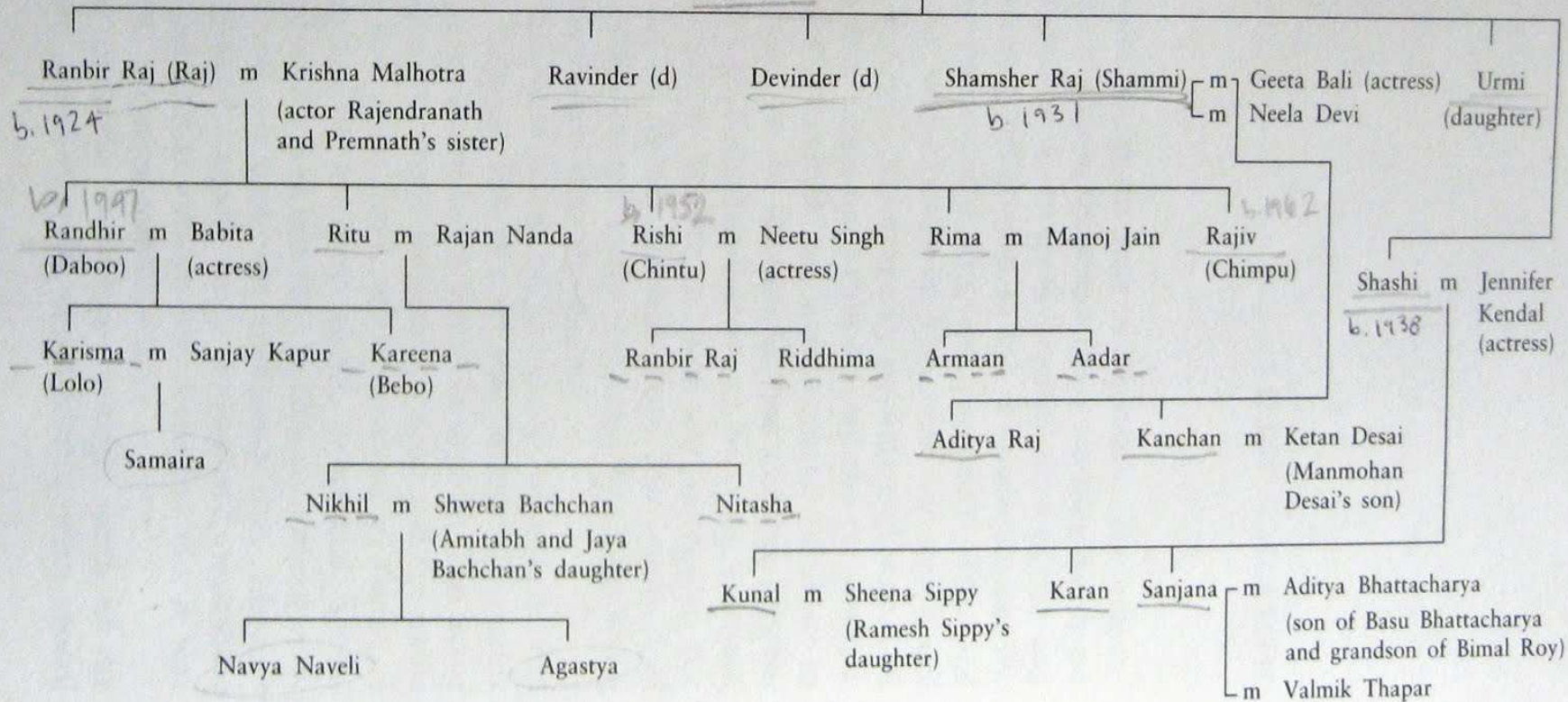


Image Courtesy: <http://www.bollywoodbollywood.com/2011/06/kapoor-family-tree.html>

NITCalicut

Comparison of Data driven and Goal driven approaches

- Data driven search takes the facts of the problem and applies the rules and legal moves to produce new facts that lead to a goal
- Goal driven search focuses on the goal, finds the rules that could produce the goal, and chain backward through successive rules and subgoals to the given facts of the problem

•

Comparison of Data driven and Goal driven approaches

Both data driven and goal driven problem solvers search the same state space graph, however, the order and actual number of states searched can differ.

The preferred strategy is determined by the properties of the problem itself

Properties include:

- **Complexity of the rules**
- **The shape of the space**
- **Nature and availability of the problem data.**

All of these properties vary for different problems

Uninformed Search

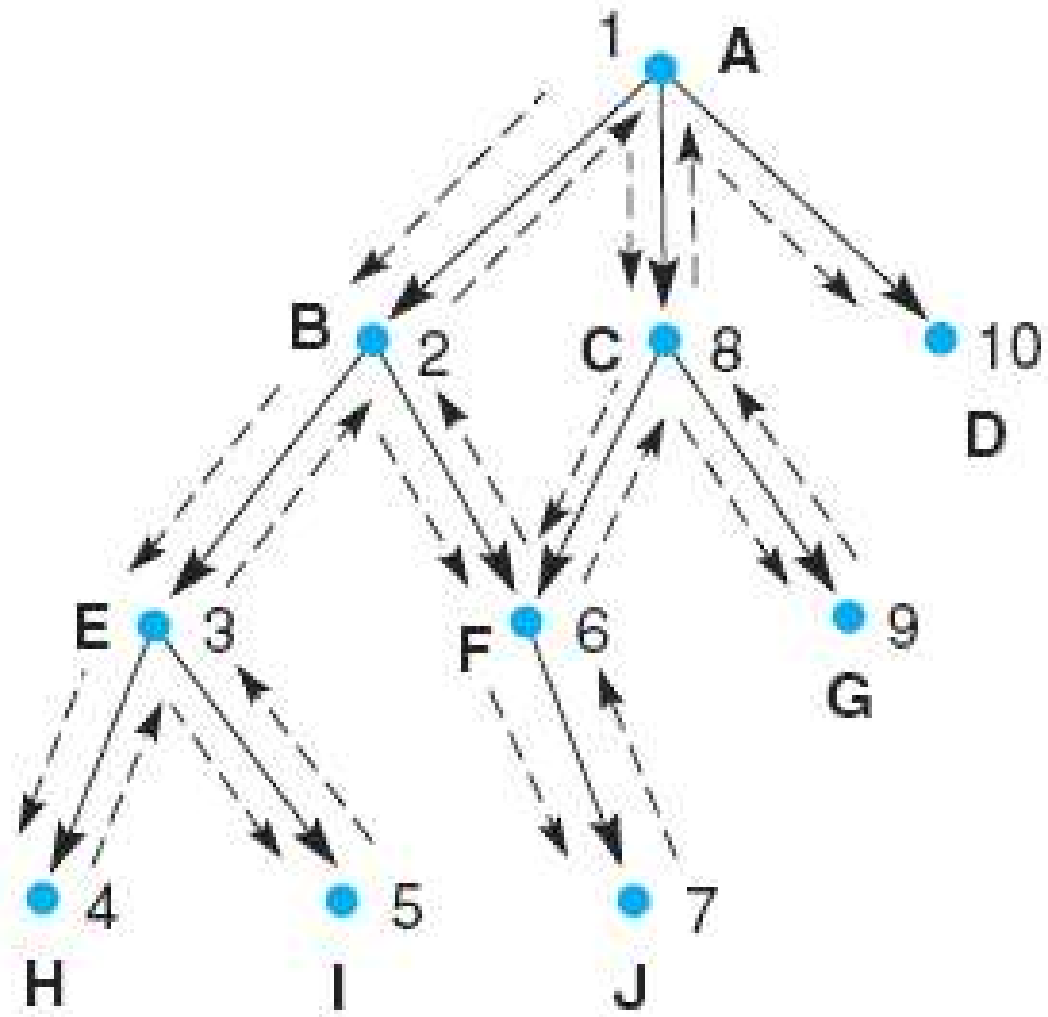
The following uninformed search techniques for searching the state space graph will be discussed in this course:

- Backtracking Search
- Breadth First Search(BFS)
- Depth First Search(DFS)
- Depth-First Search with Iterative Deepening (IDFS)

Implementing Graph Search: Function backtrack algorithm

```
function backtrack;

begin
  SL := [Start]; NSL := [Start]; DE := [ ]; CS := Start;           % initialize:
  while NSL  $\neq$  [ ] do                                           % while there are states to be tried
  begin
    if CS = goal (or meets goal description)
    then return SL;                                               % on success, return list of states in path.
    if CS has no children (excluding nodes already on DE, SL, and NSL)
    then begin
      while SL is not empty and CS = the first element of SL do
      begin
        add CS to DE;                                           % record state as dead end
        remove first element from SL;                           %backtrack
        remove first element from NSL;
        CS := first element of NSL;
      end
      add CS to SL;
    end
    else begin
      place children of CS (except nodes already on DE, SL, or NSL) on NSL;
      CS := first element of NSL;
      add CS to SL
    end
  end
end;
return FAIL;
end.
```

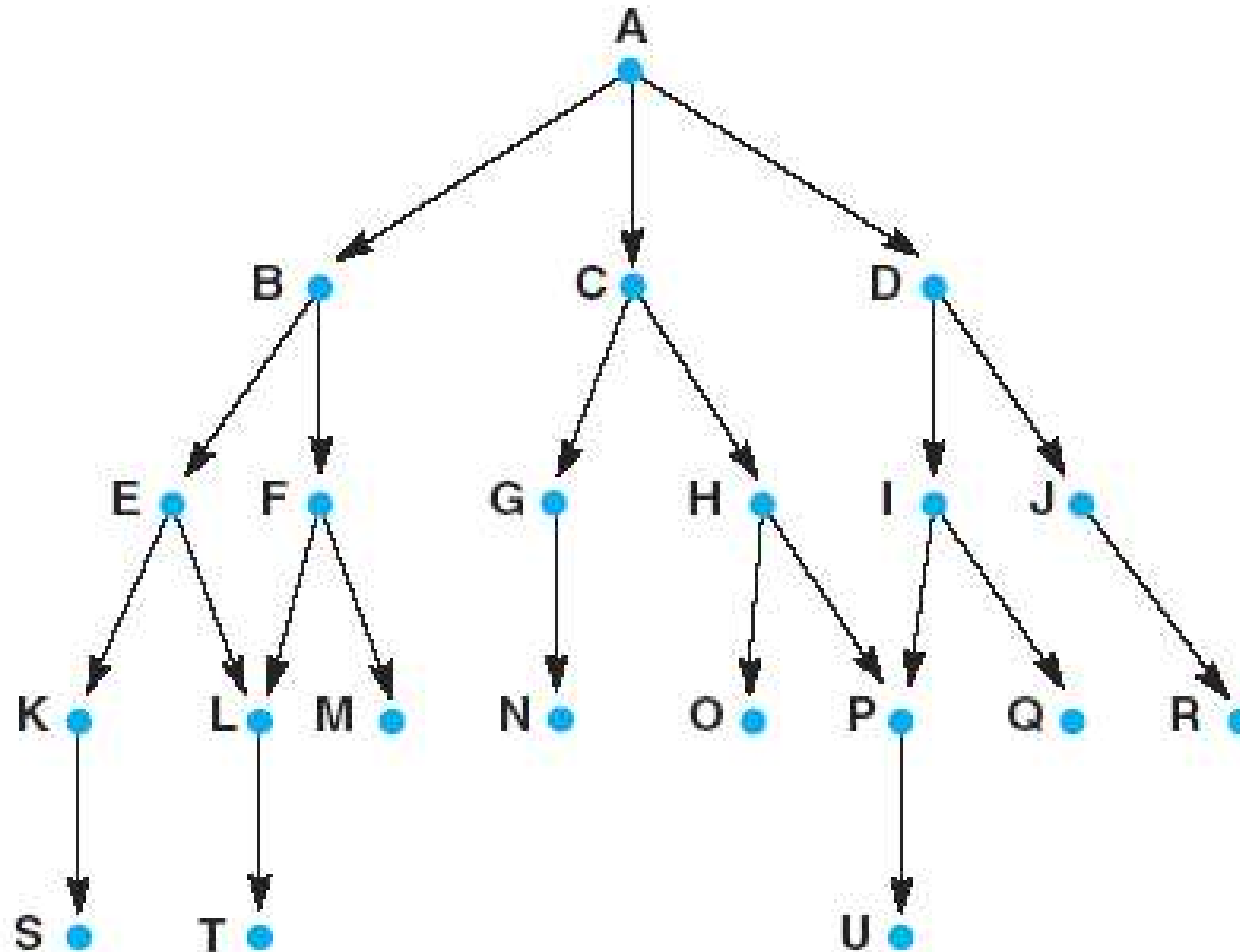


A trace of backtrack on the graph

Initialize: SL = [A]; NSL = [A]; DE = []; CS = A;

AFTER ITERATION	CS	SL	NSL	DE
0	A	[A]	[A]	[]
1	B	[B A]	[B C D A]	[]
2	E	[E B A]	[E F B C D A]	[]
3	H	[H E B A]	[H I E F B C D A]	[]
4	I	[I E B A]	[I E F B C D A]	[H]
5	F	[F B A]	[F B C D A]	[E I H]
6	J	[J F B A]	[J F B C D A]	[E I H]
7	C	[C A]	[C D A]	[B F J E I H]
8	G	[G C A]	[G C D A]	[B F J E I H]

Graph for BFS and DFS example



BFS(Breadth First Search)

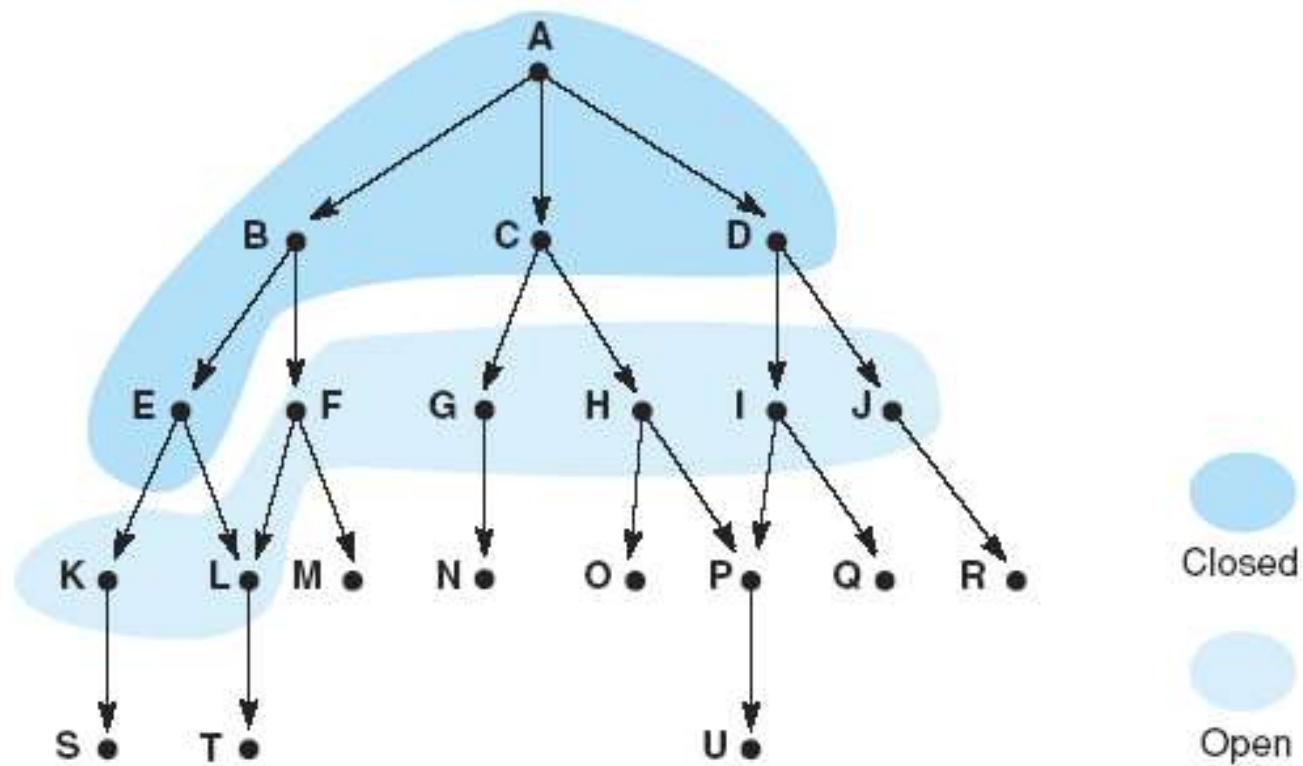
```
function breadth_first_search;

begin
    open := [Start];                                % initialize
    closed := [ ];
    while open ≠ [ ] do                             % states remain
        begin
            remove leftmost state from open, call it X;
            if X is a goal then return SUCCESS        % goal found
            else begin
                generate children of X;
                put X on closed;
                discard children of X if already on open or closed;
                put remaining children on right end of open
            end
        end
    end
    return FAIL
end.
```

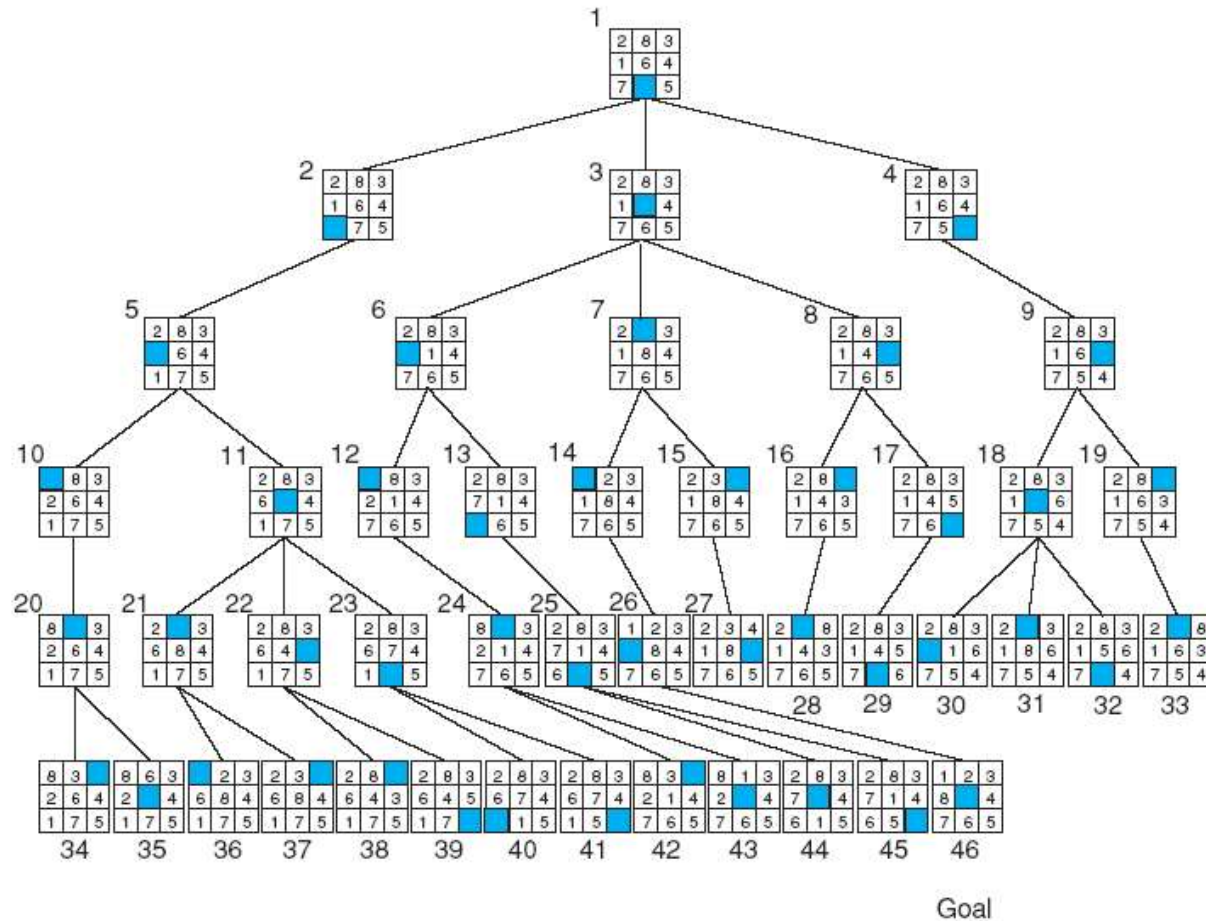
% no states left

1. **open = [A]; closed = []**
2. **open = [B,C,D]; closed = [A]**
3. **open = [C,D,E,F]; closed = [B,A]**
4. **open = [D,E,F,G,H]; closed = [C,B,A]**
5. **open = [E,F,G,H,I,J]; closed = [D,C,B,A]**
6. **open = [F,G,H,I,J,K,L]; closed = [E,D,C,B,A]**
7. **open = [G,H,I,J,K,L,M]** (as L is already on open); **closed = [F,E,D,C,B,A]**
8. **open = [H,I,J,K,L,M,N]; closed = [G,F,E,D,C,B,A]**
9. and so on until either U is found or **open = []**

open = [(D,A), (E,B), (F,B), (G,C), (H,C)]; closed = [(C,A), (B,A), (A,nil)]



BFS on 8-Puzzle



BFS

- **Breadth-First:**
- BFS examines all the nodes at level n before proceeding to level $n+1$.
- BFS always finds the shortest path to a goal node. In a problem where it is known that a simple solution exists, this solution will be found.
- Unfortunately, if there is a bad branching factor, i.e., states have a high average number of descendants, the combinatorial explosion may prevent the algorithm from finding a solution using the available space.

BFS

- **Breadth-First:**
- BFS examines all the nodes at level n before proceeding to level $n+1$.
- BFS always finds the shortest path to a goal node. In a problem where it is known that a simple solution exists, this solution will be found.
- Unfortunately, if there is a bad branching factor, i.e., states have a high average number of descendants, the combinatorial explosion may prevent the algorithm from finding a solution using the available space.
- **The space utilization of breadth-first search:**

BFS

- **Breadth-First:**
- **The space utilization of breadth-first search:**
- If each state has an average of B children, the number of states on a given level is B times the number of states on the previous level. This gives B^n states on level n .
- Breadth-first search would place all of these nodes on OPEN when it begins examining level n . This can be prohibitive if solution paths are long.

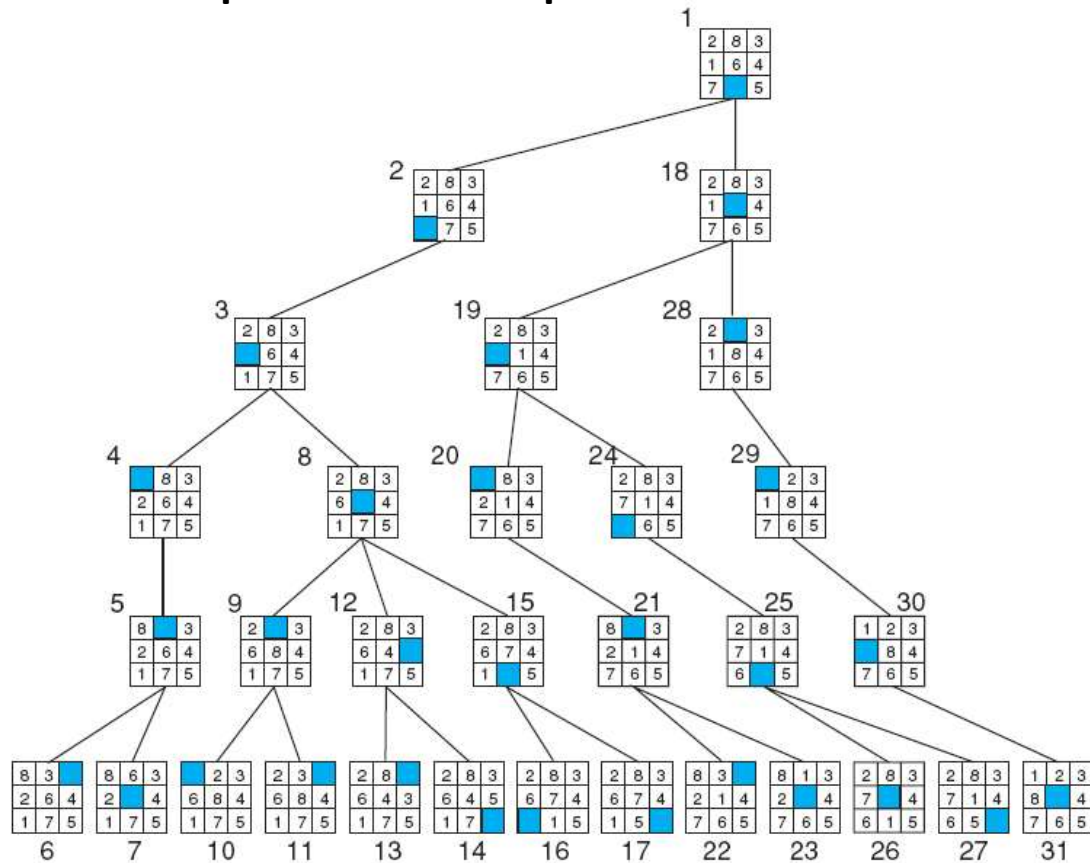
DFS(Depth First Search)

```
begin
  open := [Start];                                % initialize
  closed := [ ];
  while open ≠ [ ] do                             % states remain
    begin
      remove leftmost state from open, call it X;
      if X is a goal then return SUCCESS           % goal found
      else begin
        generate children of X;
        put X on closed;
        discard children of X if already on open or closed; % loop check
        put remaining children on left end of open      % stack
      end
    end
  end;
  return FAIL                                     % no states left
end.
```

1. **open = [A]; closed = []**
2. **open = [B,C,D]; closed = [A]**
3. **open = [E,F,C,D]; closed = [B,A]**
4. **open = [K,L,F,C,D]; closed = [E,B,A]**
5. **open = [S,L,F,C,D]; closed = [K,E,B,A]**
6. **open = [L,F,C,D]; closed = [S,K,E,B,A]**
7. **open = [T,F,C,D]; closed = [L,S,K,E,B,A]**
8. **open = [F,C,D]; closed = [T,L,S,K,E,B,A]**
9. **open = [M,C,D], as L is already on closed; closed = [F,T,L,S,K,E,B,A]**
10. **open = [C,D]; closed = [M,F,T,L,S,K,E,B,A]**
11. **open = [G,H,D]; closed = [C,M,F,T,L,S,K,E,B,A]**

DFS on 8-Puzzle

Depth-first search of the 8-puzzle with a depth bound of 5.



Goal

DFS

- If it is known that the solution path will be long, depth-first search will not waste time searching a large number of "shallow" states in the graph.
- DFS can get “lost” deep in a graph, missing shorter paths to a goal or even becoming stuck in an infinitely long path that does not lead to a goal.
- DFS is much more efficient for search spaces with many branches because it does not have to keep all the nodes at a given level on the open list.

DFS

-
- **The space utilization of depth-first search:**
- a linear function of the length of the path.
- At each level, open retains only the children of a single state.
- If a graph has an average of B children per state, this requires a total space usage of $B \times n$ states to go n levels deep into the space.

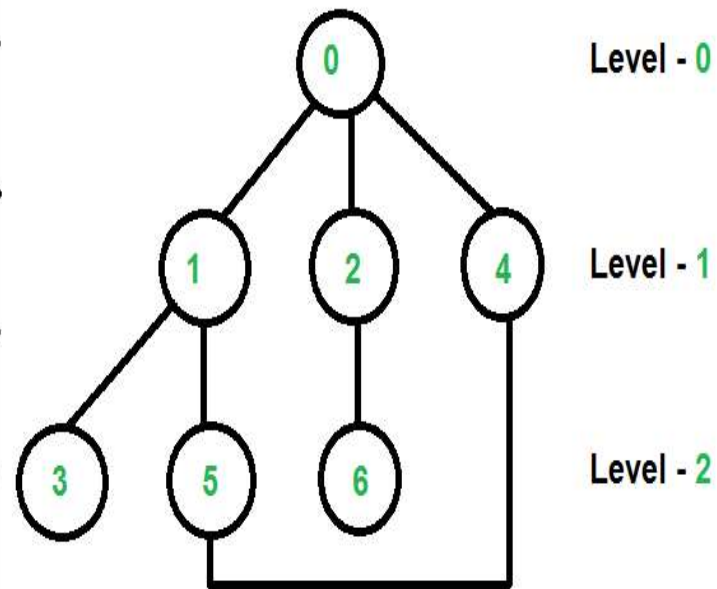
Depth-First Search with Iterative Deepening (IDFS)

Depth-first iterative deepening performs a depth-first search of the space with a depth bound of 1.

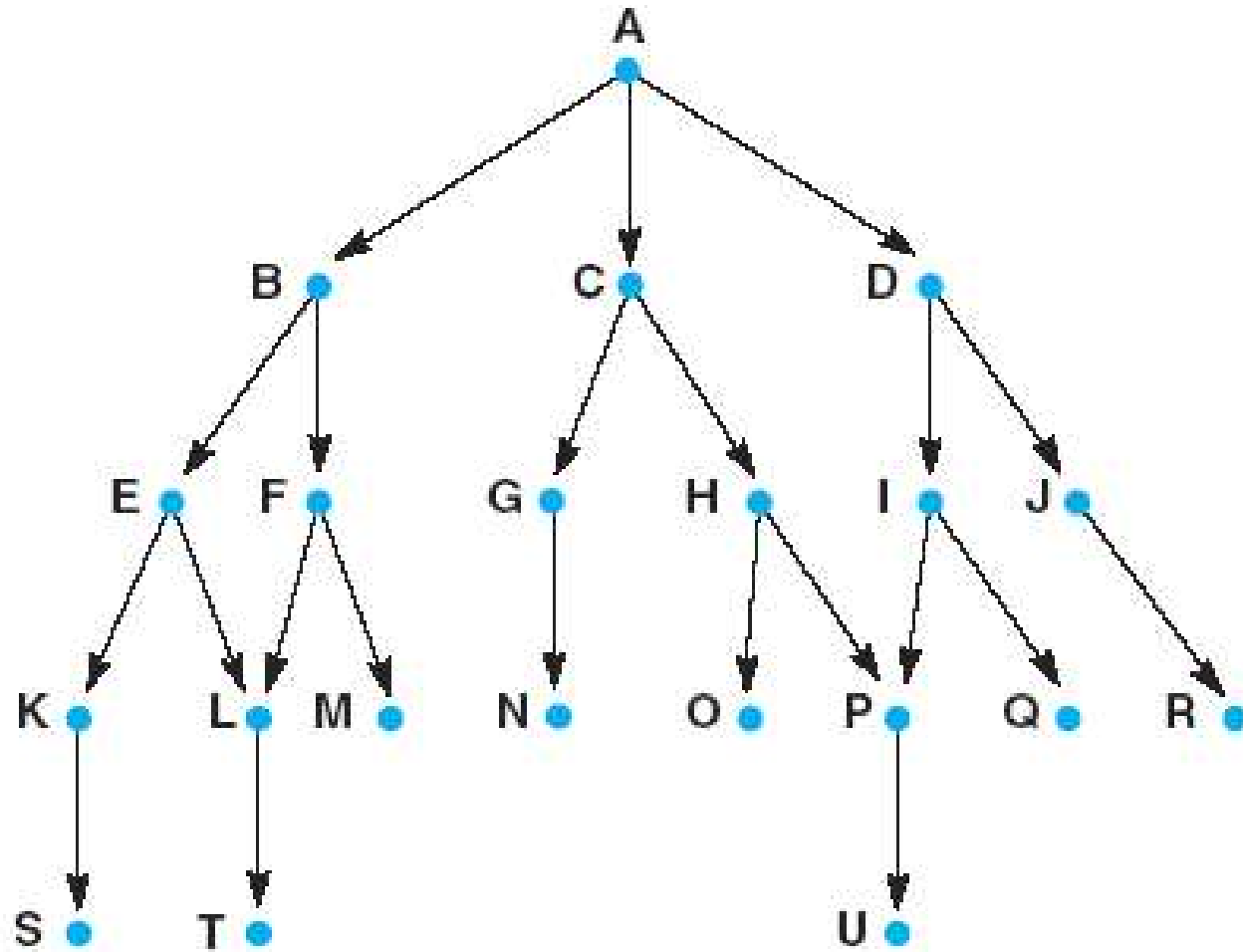
- If it fails to find a goal, it performs another depth-first search with a depth bound of 2. This continues, increasing the depth bound by one at each iteration.
- At each iteration, the algorithm performs a complete depth-first search to the current depth bound.
- No information about the state space is retained between iterations.
- It is guaranteed to find a shortest path to a goal.
- Because it does only depth-first search at each iteration, the space usage at any level n is $B \times n$, where B is the average number of children of a node.
- The time complexity is actually of the same order of magnitude as DFS and BFS: $O(B^n)$.
- Depth-first, breadth-first, and depth-first iterative deepening may be shown to have worst-case exponential time complexity. This is true for all uninformed search algorithms.

IDFS Example

Depth	Iterative Deepening Depth First Search
0	0
1	0 1 2 4
2	0 1 3 5 2 6 4 5
3	0 1 3 5 4 2 6 4 5 1

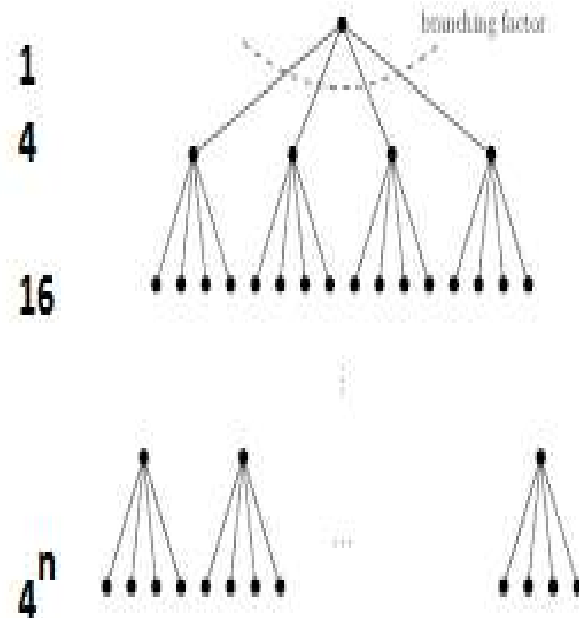


IDFS



Comparison of nodes expanded by BFS and IDFS

- For a graph with branching factor b and the shallowest goal in depth d , the no. of Nodes expanded by BFS can be in the worst case



Comparison of nodes expanded by BFS and IDFS

- For a graph with branching factor b and the shallowest goal in depth d , the no. of Nodes expanded by BFS can be in the worst case

$$N_{bf} = 1 + b + b^2 + \dots + b^d = \frac{(b^{d+1} - 1)}{b - 1}$$

The no. of nodes expanded by a complete IDFS upto a level j is

$$N_{dfj} = \frac{(b^{j+1} - 1)}{b - 1}$$

Comparison of nodes expanded by BFS and IDFS...

- In the worst case for a goal in depth d IDFS has to expand

$$N_{id} = \sum_{j=0}^d \frac{(b^{j+1} - 1)}{b - 1}$$

For large d the
Ratio N_{id}/N_{bf}
is $b/(b-1)$

For a branching
factor of 10
and deep goals
we expand only
11% more state
in IDFS.

$$= \frac{1}{b-1} \left[b \left(\sum_{j=0}^d b^j \right) - \sum_{j=0}^d 1 \right]$$

$$= \frac{1}{b-1} \left[b \left(\frac{(b^{d+1} - 1)}{b - 1} \right) - (d + 1) \right]$$

$$N_{id} = \frac{b^{d+2} - 2b - bd + d + 1}{(b-1)^2}$$